

A Single Foundation Model for Speed-Test Early Termination

Jithendra (2025MCS2743)

Nitin (2025MCS2152)
FMFSTLT Course Project

Repository: <https://github.com/g-nitin-1/FMFSTLT>

Abstract

TurboTest-style early termination uses two task-specific models: an XGBoost regressor that predicts final throughput from a prefix, and a Transformer policy that decides when a test can stop safely. We investigate whether a single foundation model pre-trained on speed-test traces can replace this two-model pipeline while preserving the same Stage 1 to Stage 2 decision flow. After patch-based, multitask, causal, and two-stage variants, our final system uses a causal bucket-level encoder, per-decision throughput heads, and a policy module that consumes the foundation model’s own Stage 1 predictions. The foundation policy matches the specialized policy closely, but the deployed system does not fully beat TurboTest because XGBoost remains stronger at prefix throughput prediction.

1 Problem and Data

TurboTest-style early termination uses Stage 1 prefix throughput prediction followed by a Stage 2 stop policy. We ask whether one pretrained foundation model can replace the separate XGBoost regressor and Transformer policy while preserving this order. We use public M-Lab-derived traces represented as 100 buckets at 100 ms resolution with 13 normalized features per bucket. The data follows a sampled-day scale: 10 train/test dates from April 2024 to January 2025 and two robustness dates from February and March 2025. The training source has 800k tests, split into 720k train UUIDs and 80k validation UUIDs; test and robustness contain about 40k and 132k tests. Baseline comparisons are frozen at $\epsilon = 10\%$ because other epsilon Stage 2 datasets were found to contain stale XGBoost predictions.

2 Models

TurboTest reproduction. Stage 1 flattens the latest 20×13 bucket window to 260 features and trains an XGBoost regressor. Stage 2 uses 20 decision points at 500 ms stride, computes a permanent-safe suffix oracle from Stage 1 errors, and trains an 8-layer Transformer stop policy. The frozen $\epsilon = 10\%$ baseline uses threshold 0.25.

Foundation model. The final foundation architecture keeps the same flow:

$$x \in \mathbb{R}^{100 \times 13} \rightarrow \text{causal bucket encoder} \rightarrow \text{per-decision throughput heads} \rightarrow \text{Stage 1 predictions} \rightarrow \text{causal Stage 2 policy} \rightarrow \text{stop probability.}$$

Table 1: Experiment story. Each step narrowed the cause of failure.

Variant	How it worked and what it showed
v1 direct	Patchified the trace into 20 patches, used a CLS-pooled Transformer classifier on each prefix, and predicted only stop/continue. A self-check pairing this classifier with the separate v1 throughput head gave only 0.265 test within- ϵ , so v1’s own reported Mbps was not reliable at stop time.
v1 throughput	The same patch encoder with a CLS regression head predicted final throughput well on val/test, but this was mostly a full-trace task and did not solve the early-prefix prediction needed by Stage 2.
v1.5/v2	Added per-decision throughput and stop heads with multitask and policy-aware losses. These runs tested closer variants of the desired system, but their own deployed checks remained weak; the best v2 self-check reached only 0.113 test within- ϵ .
v3 causal	Removed patch compression, kept all 100 buckets, and used causal next-bucket pretraining. This preserved timing information and avoided future leakage.
Final two-stage	Used v3 as a shared encoder, trained Stage 1 first, then trained Stage 2 on detached foundation Stage 1 outputs. This is the correct project architecture. Policy quality tied the baseline, but deployed accuracy lagged because foundation prefix predictions remained weaker than XGBoost.

The encoder is our third foundation encoder variant, FMNet-v3: a causal Transformer with 10 layers, $d = 256$, 8 heads, and 1024 feed-forward width, initialized from causal next-bucket pretraining. Decision states are gathered at buckets 4, 9, $\dots$, 99. Stage 1 has two heads: one emits μ_d , the predicted $\log(1 + \text{Mbps})$ final throughput at decision d , and the other emits log variance, a learned uncertainty score trained by Gaussian negative log likelihood so uncertain prefixes can be marked as less reliable. Stage 2 receives only deployable features: μ_d , log variance, elapsed time, observed-bucket count, and the decision hidden state. Labels such as relative error and stop labels are not policy inputs.

3 Experimental Progression

We did not arrive at the final architecture directly. Table 1 summarizes the main variants. The key nuance is that each intermediate model must be judged by its own deployed outputs: the stop decision and the throughput value it would actually report at that stop.

4 Training and Evaluation

The foundation run is trained in phases. Phase 1 trains the encoder and Stage 1 heads using per-decision and final log-throughput losses; for the uncertainty head we use Gaussian NLL, $\mathcal{L}_{\text{nll}} = \frac{1}{2} \{ \exp(-s_d)(y_d - \mu_d)^2 + s_d \}$, where s_d is log variance and $y_d = \log(1 + \text{Mbps}_{\text{true}})$. The final run uses cosine learning-rate decay and exponential moving average weights; this stabilized a previously noisy throughput trajectory. Phase 2 freezes the encoder and Stage 1 heads, detaches the Stage 1 out-

Table 2: Stage 1 throughput error. Prefix MAE is averaged over valid decision points; lower is better.

Subset	XGB prefix MAE	Found. prefix MAE	Found. final MAE
Val	18.98	22.96	9.68
Test	17.72	21.06	8.56
Robust	32.14	49.61	35.21

Table 3: Stage 2 early-stop results at the selected thresholds. Within is the fraction of stopped tests within 10% of final throughput.

Subset	System	F1	Pol. within	Dep. within	Save ms
Val	Turbo B+X	0.8696	0.6684	0.6684	5241
Val	Foundation	0.8703	0.6650	0.5867	5087
Test	Turbo B+X	0.8608	0.6786	0.6786	5145
Test	Foundation	0.8604	0.6734	0.5908	4968
Robust	Turbo B+X	0.8565	0.6608	0.6608	5245
Robust	Foundation	0.8552	0.6598	0.5785	5073

puts at the Stage 2 boundary, and trains only the policy module using stop-label BCE. Phase 3 end-to-end tuning was attempted but discarded because it improved policy metrics slightly while increasing Stage 1 error by about $3-4\times$.

We report two kinds of Stage 2 metrics. *Policy* metrics ask whether the chosen stop point matches the stopping target. *Deployed* metrics use the value the system would actually report at runtime. For the foundation variants, deployed evaluation means using the model’s own stop decision and its own prefix throughput prediction at that stop. This distinction matters because a stop classifier can look plausible while its paired throughput head is still too inaccurate.

5 Results

Table 2 shows that the foundation model learns a strong full-trace throughput head, but TurboTest’s XGBoost regressor remains better at the per-decision prefix predictions that matter at early stopping time.

Table 3 separates policy quality from deployed user-facing accuracy. F+X evaluates the foundation stop policy using the same XGBoost prefix predictions used by TurboTest; this isolates the stop-policy decision. F+F is the deployed foundation system: the foundation chooses the stop point and reports its own prefix throughput prediction.

The foundation policy is tied with the specialized Stage 2 Transformer when both are judged as stop policies: F1 differs by at most 0.0013 and policy-within by at most 0.0052. The self-contained checks explain why earlier variants were not selected. v1 classifier plus v1 throughput achieved only 0.2653 test deployed within- ϵ with 5720 ms savings, and the best v2 self-check achieved only 0.1134 with 4330 ms savings. The final v3 F+F system is therefore the best deployed foundation variant, reaching 0.5908 on test with 4968 ms savings.

6 What Worked and What Failed

The positive result is that the foundation architecture can learn the Stage 2 stopping behavior. The policy module is small compared with the encoder, yet it reaches essentially the same stop-policy quality as the specialized Transformer when evaluated under the same prediction source. This supports the idea that

self-supervised trace representations can transfer to network-measurement control tasks.

The negative result is equally important: the foundation model does not yet replace XGBoost as the deployed Stage 1 prefix regressor. Full-trace throughput prediction becomes strong, but early-stop deployment depends on predictions made before the test finishes. At those intermediate decision points, the specialized XGBoost window model remains better. Therefore, a claim based only on final throughput MAE would be misleading for early termination.

7 Discussion and Conclusion

The final result is not a strict replacement win, but it is informative. The single foundation model preserves the Stage 1 to Stage 2 structure and learns a competitive stop policy. The bottleneck is per-decision prefix throughput prediction, not stop-policy learning. We conclude that foundation pretraining is useful for shared representation learning and policy transfer, but it does not yet fully replace TurboTest’s specialized XGBoost prefix regressor.

For future work, the most direct next experiment is teacher distillation from the stored XGBoost prefix predictions during Stage 1 training. XGBoost would be used only as a training-time teacher, while deployment would still call only the foundation model. We stop experiments here and submit the working code with this revised interpretation.